Software Engineering Lab

Lab 16

Dependency Injection & Inversion of Control

Memisahkan Konfigurasi dari Penggunaan di Go



Lukman / lukman-ss

Hard-Coded Dependencies Mengunci Logika Bisnis pada Infrastruktur

Ketika kelas menginstansiasi dependensinya sendiri, pengujian menjadi lambat dan penggantian vendor memaksa modifikasi menyeluruh.

DAMPAK TIGHT COUPLING

Pengujian Lambat

Setiap test memerlukan setup jaringan, database, atau payment provider riil yang flaky.

DAMPAK TIGHT COUPLING

Vendor Lock-in

Pergantian payment gateway memaksa edit banyak kelas yang bergantung pada implementasi konkret.

DAMPAK TIGHT COUPLING

Bukan Berarti Malas

Tim aktif menulis kode, tetapi bukan berarti produktif jika setiap perubahan memicu domino refactoring.

Ketika sebuah kelas membangun dependensinya sendiri, logika bisnis menjadi terkunci pada infrastruktur eksternal. Pengujian unit jadi sulit karena harus mengandalkan status jaringan nyata yang tidak deterministik.

Hard-coded dependencies couple business logic to infrastructure, breaking testability and evolvability.

Code Review Manual Tidak Menangkap Dependensi Tersembunyi

Dependensi yang diinstansiasi secara internal sulit dilacak hanya dengan membaca kode secara sekilas.

KEGAGALAN APPROACH NAIF

Tidak Lengkap

Reviewer mungkin melewatkan instantiate baru di dalam method ketika fokus pada alur utama.

KEGAGALAN APPROACH NAIF

Tidak Otomatis

Manual review tidak dapat memverifikasi bahwa semua dependensi telah dipisahkan ke constructor.

KEGAGALAN APPROACH NAIF

Skala Buruk

Pada codebase besar, jejak dependensi tersebar di banyak file; bukan berarti review manual cukup.

Code review manual mengandalkan perhatian manusia untuk menangkap pola coupling. Pendekatan ini tidak otomatis menjamin bahwa setiap dependensi eksternal telah diinjeksi melalui constructor.

Manual review cannot systematically enforce dependency separation across a large codebase.

Pemisahan Konfigurasi dari Penggunaan

Fase konfigurasi (composition root) terpisah dari fase penggunaan (business logic).

COMPOSITION ROOT

1. Konfigurasi

Instansiasi implementasi konkret dan penyuntikan ke dalam constructor (main.go).

BUSINESS LOGIC

2. Penggunaan

Kelas menerima dependensi melalui parameter constructor; tidak mengetahui implementasi konkret.

Prinsip inti Dependency Injection adalah memisahkan pembentukan objek dari penggunaannya. Composition root bertugas merakit object graph, sementara modul bisnis hanya menggunakan dependensi yang telah tersedia.

Separate configuration (composition root) from use (business logic) to achieve loose coupling.

Alur Kerja Dependency Injection Manual

Dari instansiasi gateway hingga delegasi ke method Charge.

MAIN.GO

1. Composition Root

Membuat realGateway := &di.RealGateway{}.

NEWPROCESSOR

2. Constructor Injection

processor := di.NewProcessor(realGateway); gateway tersimpan sebagai field unexported.

PROCESSPAYMENT

3. Validasi Input

Periksa amount <= 0; kembalikan error jika invalid.

GATEWAY.CHARGE

4. Delegasi

Buat Money value object dan panggil gateway.Charge(m).

ERROR RETURN

5. Propagasi Error

Error dari gateway dipropagasikan ke caller tanpa ditelan.

Pipeline DI manual dimulai dari composition root yang menginialisasi implementasi konkret. Konstruktor menyimpan dependensi sebagai field, method bisnis memvalidasi input lalu mendelegasikan ke gateway.

Manual wiring sequence: instantiate → inject → validate → delegate → propagate error.

cmd/demo/main.go:16–36

Constructor Injection: NewProcessor Menerima Interface

Processor menyimpan gateway PaymentGateway tanpa mengetahui implementasinya.

```
type Processor struct {
    gateway PaymentGateway
}

func NewProcessor(g PaymentGateway) *Processor {
    return &Processor{gateway: g}
}

func (p *Processor) ProcessPayment(amount int) error {
    if amount <= 0 {
        return errors.New("invalid amount")
    }
    m := Money{Amount: amount, Currency: "USD"}
    return p.gateway.Charge(m)
}
```

Constructor injection membuat Processor hanya mengenal interface PaymentGateway, bukan implementasi konkret RealGateway. Dependensi dibuat eksplisit di signature constructor.

Constructor injection membuat dependensi eksplisit saat konstruksi.

Service Locator Menyembunyikan Dependensi Sesungguhnya

BadProcessor menerima Container alih-alih PaymentGateway, membuat dependensi tersembunyi.

SKENARIO KEGAGALAN LOKATOR

Dependensi Tersembunyi

Constructor hanya menerima Container; developer tidak tahu method butuh PaymentGateway.

SKENARIO KEGAGALAN LOKATOR

Pencarian Dinamis

ProcessPayment memanggil p.container.GetPaymentGateway() saat runtime, menambah indirection.

SKENARIO KEGAGALAN LOKATOR

Beban Mocking Tambahan

Pengujian unit terpaksa menyusun MockContainer bertingkat hanya untuk menyuplai gateway konkret.

SKENARIO KEGAGALAN LOKATOR

Kopling Container Kaku

BadProcessor terikat langsung pada interface Container, bukan abstraksi domain spesifik.

Service Locator menyembunyikan dependensi nyata di balik lookup method container saat runtime. Hal ini menyulitkan pelacakan dependensi dan membebani pengujian unit karena developer wajib menyiapkan MockContainer bertingkat hanya untuk menyuplai gateway konkret.

Service Locator obscures dependencies, complicating testing and reducing code transparency.

Kontras Constructor Injection vs Service Locator

Perbandingan mekanisme, pengujian, dan konsekuensi arsitektur.

Aspek	Constructor Injection	Service Locator
Dependensi	Eksplisit di constructor	Tersembunyi di container
Mocking	Langsung inject mock	Harus mock container + get()
Kontrak Dependensi	Membuat dependensi eksplisit saat konstruksi	Tersembunyi di balik lookup container
Kompleksitas	Mudah dipahami	Menambah indirection layer
Standar Industri	Diterima sebagai best practice	Dipandang sebagai anti-pattern

Tabel ini memetakan perbedaan mendasar antara dua pendekatan. Constructor injection menghasilkan dependensi yang eksplisit dan mudah diuji; service locator memperkenalkan lapisan tambahan yang menyulitkan isolasi.

Compare injection mechanisms along axes of visibility, testability, and industry acceptance.

Keputusan Desain DI pada Lab Ini

Tiga keputusan arsitektural yang menentukan struktur kode.

KEPUTUSAN PENTING

Pilih Constructor Injection

Default modern; membantu menjaga kontrak dependensi tetap terlihat dan eksplisit.

KEPUTUSAN PENTING

Hindari Service Locator di Domain

Container hanya digunakan di composition root; tidak disuntikkan ke service.

KEPUTUSAN PENTING

Gunakan Value Object Langsung

Money diinstansiasi langsung karena tidak memiliki dependensi infrastruktur.

> Lab ini memilih constructor injection sebagai pola default karena membantu menjaga kontrak dependensi tetap terlihat. Service Locator hanya digunakan untuk mendemonstrasikan anti-pattern, bukan sebagai solusi arsitektur utama.

Choose constructor injection to make dependencies explicit at construction time.

6 Test Case Lolos dengan Race Detector

Semua test berjalan dalam 0.00s per kasus tanpa dependency eksternal.

```
=== RUN   TestProcessor_Success
--- PASS: TestProcessor_Success (0.00s)
=== RUN   TestProcessor_GatewayError
--- PASS: TestProcessor_GatewayError (0.00s)
=== RUN   TestProcessor_InvalidAmount
--- PASS: TestProcessor_InvalidAmount (0.00s)
=== RUN   TestBadProcessor_Success
--- PASS: TestBadProcessor_Success (0.00s)
=== RUN   TestBadProcessor_GatewayError
--- PASS: TestBadProcessor_GatewayError (0.00s)
=== RUN   TestBadProcessor_InvalidAmount
--- PASS: TestBadProcessor_InvalidAmount (0.00s)
PASS
ok      lab16/tests   (cached)
```

Output terminal membuktikan seluruh enam test case lulus termasuk race detector. Test berhasil mengisolasi Processor dari gateway nyata, memverifikasi happy path, error propagation, dan validasi input.

Executable test suite proves isolation, error handling, and input validation without external infrastructure.

Batasan Bukti Lab: Yang Dibuktikan vs Belum

Menegaskan cakupan verifikasi terhadap skenario produksi.

BATAS PEMBUKTIAN SISTEM

YANG DIBUKTIKAN

Isolasi mock, propagation error, validasi input, race safety pada 6 test case.

BATAS PEMBUKTIAN SISTEM

YANG TIDAK DIBUKTIKAN

Lifecycle management (singleton/scoped/transient), performance overhead container, pengurangan defect rate.

BATAS PEMBUKTIAN SISTEM

BATAS KLAIM EMPIRIS

Tidak ada data kuantitatif tentang kecepatan pengujian atau stabilitas sistem setelah penerapan DI.

Lab ini membuktikan sifat isolasi dan propagasi error pada skenario sederhana. Namun belum membuktikan aspek lifecycle, kinerja, atau dampak empiris terhadap metrik rekayasa perangkat lunak.

Strictly separate proven unit-test behavior from unproven production-scale claims.

content/01-content-brief.md:13-17

Trade-off Penerapan DI Manual vs Framework

Manual wiring menambah boilerplate tetapi memperjelas kontrak tipe tanpa dependensi eksternal.

KOMPROMI REKAYASA DI

Boilerplate vs Kemudahan

Manual wiring memerlukan kode ekstra di main.go; framework mengurangi boilerplate tetapi menambah kompleksitas.

KOMPROMI REKAYASA DI

Type Safety vs Fleksibilitas

Compiler memeriksa tipe parameter, bukan seluruh validitas graph; framework refleksi lebih fleksibel tetapi menambah risiko runtime.

KOMPROMI REKAYASA DI

Tidak Otomatis Skalabel

Pada object graph sangat besar, manual wiring memerlukan ketelitian perakitan; bukan berarti framework refleksi selalu diperlukan.

Penerapan DI manual menuntut penulisan composition root yang jelas. Pada skala besar, overhead boilerplate dapat bertambah, namun trade-off ini sering diterima demi kejelasan kontrak dan kemudahan pengujian.

DI manual wiring trades boilerplate for type safety and zero external dependencies.

Race Safety pada Implementasi DI

Test dengan race detector memastikan tidak ada shared mutable state.

```
func TestProcessor_Success(t *testing.T) {
    mock := &MockGateway{}
    proc := di.NewProcessor(mock)
    err := proc.ProcessPayment(50)
    if err != nil {
        t.Fatalf("expected nil error, got %v", err)
    }
    // Race detector passes; no shared mutable state
}
```

Race detector Go memeriksa akses memori konkuren. Pada pengujian ini, skenario test ini tidak menunjukkan race pada jalur yang dieksekusi karena komponen bersifat stateless.

Skenario test ini tidak menunjukkan race pada jalur yang dieksekusi.

tests/processor_test.go:23–35

Transformasi: Dari Hard-Coded ke Constructor Injection

Perbandingan kode sebelum dan sesudah penerapan DI.

SEBELUM (TIGHT COUPLING) VS SESUDAH (DI)

Sebelum: Hard-Coded

Processor membuat RealGateway di dalam method; tidak bisa di-mock tanpa mengubah kode sumber.

SEBELUM (TIGHT COUPLING) VS SESUDAH (DI)

Sesudah: DI Manual

NewProcessor menerima PaymentGateway; mock disuntikkan langsung pada test.

SEBELUM (TIGHT COUPLING) VS SESUDAH (DI)

Bukan Berarti Sempurna

DI bukan solusi ajaib; tetap perlu disiplin composition root dan hindari over-injection.

Sebelum DI, kelas bisnis langsung menginstansiasi dependensi konkret, mengunci pada satu implementasi. Setelah DI, dependensi datang dari luar, memungkinkan penggantian dan pengujian terisolasi.

Replace internal instantiation with external injection to achieve loose coupling.

Interface PaymentGateway: Kontrak Tunggal

Definisi metode Charge yang menerima value object Money.

```
type Money struct {
    Amount   int
    Currency string
}

type PaymentGateway interface {
    Charge(m Money) error
}
```

Interface PaymentGateway mendefinisikan kontrak tunggal untuk semua implementasi gateway. Money adalah value object yang mewakili uang dengan amount dan currency.

Define minimal interfaces that capture the required behavior without exposing implementation details.

Money: Value Object yang Diinstansiasi Langsung

Value object tidak memerlukan DI karena tidak memiliki dependensi eksternal.

```
func (p *Processor) ProcessPayment(amount int) error {
    if amount <= 0 {
        return errors.New("invalid amount")
    }
    m := Money{Amount: amount, Currency: "USD"}
    return p.gateway.Charge(m)
}
```

Money dibuat langsung di dalam method karena merupakan struktur data murni tanpa dependensi infrastruktur. Menggunakan DI untuk value object akan menambah abstraksi tanpa manfaat.

Instantiate value objects directly; reserve DI for services that depend on external infrastructure.

internal/di/processor.go:15–21

MockGateway: Test Double untuk Isolasi Pengujian

Mock meniru PaymentGateway tanpa panggilan jaringan nyata.

```
type MockGateway struct {
    ChargedMoney di.Money
    ShouldFail   bool
}

func (m *MockGateway) Charge(money di.Money) error {
    if m.ShouldFail {
        return errors.New("gateway unavailable")
    }
    m.ChargedMoney = money
    return nil
}
```

MockGateway merekam panggilan Charge dan dapat dikonfigurasi untuk gagal. Hal ini memungkinkan pengujian error propagation tanpa koneksi jaringan riil.

Use mock doubles to isolate unit tests from external side effects.

tests/processor_test.go:10–21

Composition Root: main.go sebagai Titik Penyambungan

Satu-satunya tempat yang tahu implementasi konkret dan menyuntikkannya.

```
func main() {
    realGateway := &di.RealGateway{}
    processor := di.NewProcessor(realGateway)
    fmt.Println("--- Running Constructor Injection ---")
    err := processor.ProcessPayment(100)
    if err != nil {
        fmt.Printf("Error: %v\n", err)
    }
}
```

Composition root adalah fungsi main yang bertanggung jawab merakit object graph. Ia satu-satunya tempat yang mengimpor implementasi konkret RealGateway.

Centralize concrete instantiation in a single composition root to maintain separation of concerns.

cmd/demo/main.go:16–26

Validasi Input Sebelum Delegasi ke Gateway

Processor menolak amount <= 0 tanpa memanggil gateway.

```
func (p *Processor) ProcessPayment(amount int) error {
    if amount <= 0 {
        return errors.New("invalid amount")
    }
    // ... create Money and call gateway
}
```

Validasi input dilakukan sebelum delegasi ke dependensi eksternal. Ini mencegah panggilan jaringan sia-sia dan memastikan hanya amount yang valid yang diproses.

Validate inputs early to fail fast and avoid side effects in downstream dependencies.

Checklist Penerapan Dependency Injection

Standar wajib sebelum menggabungkan modul bisnis dengan dependensi eksternal.

- ✓ Dependensi dideklarasikan sebagai interface, bukan tipe konkret.
- ✓ Constructor injection digunakan sebagai default untuk semua service.
- ✓ Composition root (main.go atau setara) adalah satu-satunya tempat yang membuat implementasi konkret.
- ✓ Tidak ada container.Get(...) di dalam logika domain.
- ✓ Value objects (Money, DateTime, Address) diinstansiasi langsung.
- ✓ Setiap service dapat diuji dengan mock tanpa setup infrastruktur.
- ✓ Validasi input dilakukan sebelum delegasi ke dependensi eksternal.
- ✓ Test mencakup happy path, error propagation, dan invalid input.
- ✓ go test -race ./... PASS tanpa warning.
- ✓ Heuristik lab (seperti 12 parameter) tidak dikutip sebagai standar universal.

Enforce dependency injection guidelines through a systematic checklist.

Anti-Pattern Umum dalam Dependency Injection

Kesalahan desain yang sering merusak manfaat DI.

HINDARI KESALAHAN INI

Service Locator di Domain

Menyuntikkan container ke kelas bisnis; dependensi menjadi tersembunyi.

HINDARI KESALAHAN INI

Mengabstraksi Value Object

Memasukkan Money ke dalam container DI padahal hanya struktur data.

HINDARI KESALAHAN INI

Constructor Bloat

Konstruktor dengan banyak parameter menandakan kelas melakukan terlalu banyak hal.

HINDARI KESALAHAN INI

DI Overuse

Menggunakan DI untuk segala hal, termasuk fungsi murni tanpa state.

Service Locator adalah anti-pattern karena menyembunyikan dependensi. Value object seharusnya tidak diinject. Constructor bloat menunjukkan pelanggaran Single Responsibility Principle.

Identify and avoid common DI anti-patterns to preserve testability and clarity.

Reproduksi Mandiri di Mesin Lokal Menggunakan Go

Verifikasi mandiri DI dengan menjalankan test suite.

```
cd labs/16-dependency-injection
go test -v ./...    # jalankan semua test case
go test -race ./... # skenario test ini tidak menunjukkan race pada jalur yang dieksekusi
go run ./cmd/demo   # eksekusi demo constructor injection & service locator
```

Semua kode ditulis dengan Go standard library tanpa dependensi eksternal. Test suite mencakup isolasi, error propagation, dan validasi input.

Use minimal reproduction commands to verify DI implementation locally.

README.md:14-23

Prinsip Inti: Dependency Harus Eksplisit

Konfigurasi terpisah dari penggunaan; interface sebagai kontrak.

TIGA PILAR ARSITEKTUR DI

Separation of Concerns

Configuration (composition root) terpisah dari use (business logic).

TIGA PILAR ARSITEKTUR DI

Program to Interfaces

Consumer bergantung pada kontrak interface, bukan implementasi konkret.

TIGA PILAR ARSITEKTUR DI

Explicit over Implicit

Semua dependensi dinyatakan eksplisit di constructor; tidak ada magic lookup.

Prinsip DI menekankan kejelasan dependensi. Dengan memisahkan konfigurasi dan penggunaan, sistem menjadi lebih mudah diuji, dipahami, dan dikembangkan.

Architect systems with explicit dependencies and separated concerns for long-term maintainability.

Dependency Injection adalah tentang kejelasan, bukan kerumitan.

Dependency Injection separates concerns, making systems testable and evolvable.

Software Engineering Lab 16 – Dependency Injection & Inversion of Control

internal/di/gateway.go • internal/di/processor.go • internal/di/locator.go
cmd/demo/main.go • tests/processor_test.go
content/02-master-draft.md • content/01-content-brief.md

REPOSITORY

`github.com/lukman-ss/software-engineering-lab`

`labs/16-dependency-injection`

Lukman / lukman-ss

Commit: 3d936ba • Repositori: software-engineering-lab
Test passing: 6 test scenarios (success, error propagation, invalid amount)
Race detector: Skenario test ini tidak menunjukkan race pada jalur yang dieksekusi (PASS)
Zero external dependencies: 100% Go standard library.